Programowanie Zaawansowane BST

Generated by Doxygen 1.18.0

1 README 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Class Documentation 7

4.1 BST Class Reference . . . 7

4.1.1 Detailed Description . . . 8

4.1.2 Member Function Documentation . . . 8

4.1.2.1 addNode() . . . 8

4.1.2.2 findElement() . . . 8

4.1.2.3 findPath() . . . 9

4.1.2.4 preorderValues() . . . 9

4.1.2.5 printTree() . . . 9

4.1.2.6 removeNode() . . . 10

4.1.2.7 removeTree() . . . 10

4.1.2.8 saveToTextFile() . . . 11

4.1.2.9 show() . . . 11

4.2 BSTFile Class Reference . . . 11

4.2.1 Detailed Description . . . 12

4.2.2 Member Function Documentation . . . 12

4.2.2.1 loadBinary() . . . 12

4.2.2.2 loadText() . . . 12

4.2.2.3 saveBinary() . . . 13

4.3 Node Struct Reference . . . 13

4.3.1 Detailed Description . . . 14

4.3.2 Constructor & Destructor Documentation . . . 14

4.3.2.1 Node() . . . 14

5 File Documentation 15

5.1 BST.cpp File Reference . . . 15

5.1.1 Detailed Description . . . 15

5.2 BST.h File Reference . . . 15

5.2.1 Detailed Description . . . 16

5.3 BST.h . . . 16

5.4 BSTFile.cpp File Reference . . . 16

5.4.1 Detailed Description . . . 17

5.5 BSTFile.h File Reference . . . 17

5.5.1 Detailed Description . . . 17

5.6 BSTFile.h . . . 17

5.7 main.cpp File Reference . . . 18

5.7.1 Detailed Description 18
5.7.2 Function Documentation 18
5.7.2.1 main() 18

Index **19**

Chapter 1

README

Algorytm drzewa BST

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BST
: Binarne drzewo poszukiwan oparte na std::vector 7

BSTFile
: Udostepnia binarny zapis i odczyt drzewa oraz import liczb z pliku tekstowego 11

Node
: Wezel drzewa przechowywanego w kontenerze STL 13

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

BST.cpp
Implementacja binarnego drzewa poszukiwan przechowywanego w wektorze 15
BST.h
Deklaracje wezla i klasy binarnego drzewa poszukiwan 15
BSTFile.cpp
Implementacja binarnego zapisu i odczytu drzewa oraz importu liczb z pliku tekstowego 16
BSTFile.h
Deklaracje operacji plikowych dla binarnego drzewa poszukiwan 17
main.cpp
Interfejs konsolowy do obslugi binarnego drzewa poszukiwan 18

Chapter 4

Class Documentation

4.1 BST Class Reference

Binarne drzewo poszukiwan oparte na std::vector.

`#include <BST.h>`

Public Member Functions

- **BST** ()

  Tworzy puste drzewo.
- bool saveToTextFile (const std::string &fileName) const

  Zapisuje wartosci drzewa w kolejnosci preorder, po jednej liczbie w wierszu.
- bool addNode (int value)

  Dodaje wartosc, jezeli jeszcze nie wystepuje.
- bool removeNode (int value)

  Usuwa wezel o podanej wartosci.
- void removeTree ()

  Usuwa wszystkie wezly drzewa.
- void **preorder** ()

  Wypisuje wartosci w kolejnosci preorder (wezel, lewe poddrzewo, prawe poddrzewo).
- void **inorder** ()

  Wypisuje wartosci w kolejnosci inorder, czyli w porzadku rosnacym.
- void **postorder** ()

  Wypisuje wartosci w kolejnosci postorder (lewe poddrzewo, prawe poddrzewo, wezel).
- std::vector< int > preorderValues () const

  Zbiera wartosci drzewa przez iteracyjne przejscie preorder.
- void printTree () const

  Wypisuje rysunek drzewa z etykietami wezlow i lacznikami do dzieci.
- bool findElement (int value) const

  Sprawdza, czy drzewo zawiera podana wartosc.
- std::vector< int > findPath (int value) const

  Wyszukuje sciezke od korzenia do podanego elementu.
- void show ()

  Wypisuje wartosci w kolejnosci przechowywania w wektorze.

4.1.1 Detailed Description

Binarne drzewo poszukiwan oparte na std::vector.

Lewe poddrzewo zawiera wartosci mniejsze od wartosci wezla, a prawe poddrzewo wartosci wieksze. Duplikaty sa pomijane.

4.1.2 Member Function Documentation

4.1.2.1 addNode()

```
bool BST::addNode (
            int value)
```

Dodaje wartosc, jezeli jeszcze nie wystepuje.

Parameters

value	Wartosc nowego wezla.

Returns

true po dodaniu, false jesli wartosc juz istnieje.

Wzrost rozmiaru wektora oznacza, ze wstawienie utworzylo nowy wezel.

4.1.2.2 findElement()

```
bool BST::findElement (
            int value) const
```

Sprawdza, czy drzewo zawiera podana wartosc.

Parameters

value	Szukana wartosc.

Returns

`true` po znalezieniu wartosci albo `false` w przeciwnym razie.

Porownanie wybiera jedyne poddrzewo, w ktorym moze byc szukana wartosc.

4.1.2.3 findPath()

```
std::vector< int > BST::findPath (
            int value) const
```

Wyszukuje sciezke od korzenia do podanego elementu.

Parameters

value	Szukana wartosc.

Returns

Wartosci od korzenia do szukanego wezla wlacznie. Dla pustego drzewa lub braku elementu zwraca pusty wektor.

Zlozonosc

Czas i dodatkowa pamiec: O(h), gdzie h to wysokosc drzewa.

Zapis wartosci przed sprawdzeniem dopasowania wlacza znaleziony wezel do sciezki.

4.1.2.4 preorderValues()

```
std::vector< int > BST::preorderValues () const
```

Zbiera wartosci drzewa przez iteracyjne przejscie preorder.

Returns

Wektor wartosci w kolejnosci preorder albo pusty wektor dla pustego drzewa.

Prawe dziecko trafia na stos przed lewym. Poniewaz zdejmowany jest ostatni dodany indeks, lewe poddrzewo jest odwiedzane jako pierwsze.

4.1.2.5 printTree()

```
void BST::printTree () const
```

Wypisuje rysunek drzewa z etykietami wezlow i lacznikami do dzieci.

Note

Dla pustego drzewa wypisuje komunikat.

4.1.2.6 removeNode()

```
bool BST::removeNode (
            int value)
```

Usuwa wezel o podanej wartosci.

Parameters

value	Wartosc wezla przeznaczonego do usuniecia.

Returns

true po usunieciu, false jesli wartosci nie znaleziono.

Obsluguje lisc, jedno dziecko, dwoje dzieci oraz korzen. Dla dwojga dzieci wykorzystuje minimum prawego poddrzewa. Po usunieciu elementu wektora aktualizuje indeksy polaczen.

Note

Brak wartosci lub puste drzewo nie powoduje zmian.

Zlozonosc

Udane usuniecie: O(n). Brak wartosci: O(h). Dodatkowa pamiec: O(1). n oznacza liczbe wezlow, a h wysokosc drzewa.

Nastepnik w porzadku inorder ma w tym przypadku co najwyzej jedno dziecko. Po skopiowaniu jego wartosci wspolny krok przepiecia potomka usuwa nastepnik.

Po usunieciu elementu kazdy kolejny wezel w wektorze przesuwa sie o jedno miejsce.

4.1.2.7 removeTree()

```
void BST::removeTree ()
```

Usuwa wszystkie wezly drzewa.

Czysci wektor i ustawia indeks korzenia na -1. Mozna wywolywac wielokrotnie, takze dla pustego drzewa. Po wyczyszczeniu mozna ponownie dodawac elementy.

Note

Pojemnosc wektora pozostaje do ponownego wykorzystania.

Zlozonosc

Czas: O(n). Dodatkowa pamiec: O(1).

4.1.2.8 saveToTextFile()

```
bool BST::saveToTextFile (
            const std::string & fileName) const
```

Zapisuje wartosci drzewa w kolejnosci preorder, po jednej liczbie w wierszu.

Parameters

fileName	Sciezka do pliku tekstowego tworzonego lub nadpisywanego przy zapisie.

Returns

`true` po pomyslnym zapisie albo `false` w razie bledu otwarcia, zapisu lub zamkniecia pliku.

Wstawienie wartosci w tej kolejnosci do pustego drzewa odtwarza ksztalt zapisanego drzewa.

4.1.2.9 show()

```
void BST::show ()
```

Wypisuje wartosci w kolejnosci przechowywania w wektorze.

Note

Nie realizuje przejsc preorder, inorder ani postorder.

The documentation for this class was generated from the following files:

- BST.h
- BST.cpp

4.2 BSTFile Class Reference

Udostepnia binarny zapis i odczyt drzewa oraz import liczb z pliku tekstowego.

```
#include <BSTFile.h>
```

Static Public Member Functions

- static bool saveBinary (const BST &tree, const std::string &fileName)

  Zapisuje drzewo do pliku binarnego w kolejnosci preorder.
- static bool loadBinary (BST &tree, const std::string &fileName)

  Wczytuje drzewo z pliku zapisanego przez saveBinary.
- static bool loadText (BST &tree, const std::string &fileName)

  Dodaje do drzewa liczby calkowite z pliku tekstowego.

4.2.1 Detailed Description

Udostepnia binarny zapis i odczyt drzewa oraz import liczb z pliku tekstowego.

4.2.2 Member Function Documentation

4.2.2.1 loadBinary()

```
bool BSTFile::loadBinary (
            BST & tree,
            const std::string & fileName)  [static]
```

Wczytuje drzewo z pliku zapisanego przez saveBinary.

Parameters

out	*tree*	Drzewo zastepowane wynikiem po poprawnym odczycie calego pliku.
	fileName	Sciezka do pliku binarnego.

Returns

`true` po wczytaniu albo `false` przy bledzie odczytu lub niepoprawnych danych.

Odrzuca niepelne dane, powtorzone wartosci i nadmiarowe bajty. Wstawienie wartosci preorder do pustego drzewa odtwarza ksztalt zapisanego drzewa. Drzewo docelowe otrzymuje wynik dopiero po sprawdzeniu konca pliku.

4.2.2.2 loadText()

```
bool BSTFile::loadText (
            BST & tree,
            const std::string & fileName)  [static]
```

Dodaje do drzewa liczby calkowite z pliku tekstowego.

Parameters

in, out	*tree*	Drzewo uzupelniane po poprawnym odczycie calego pliku.
	fileName	Sciezka do pliku tekstowego.

Returns

`true` po wczytaniu albo `false` przy bledzie odczytu lub niepoprawnej liczbie.

Biale znaki oddzielaja liczby. Powtorzone wartosci sa pomijane. Kazda liczba musi miescic sie w zakresie typu `int`. Kopia pozwala zatwierdzic wszystkie dodane wartosci dopiero po walidacji pliku.

Sprawdzenie konca tokena odrzuca wpisy z dodatkowymi znakami po liczbie.

4.2.2.3 saveBinary()

```
bool BSTFile::saveBinary (
            const BST & tree,
            const std::string & fileName)  [static]
```

Zapisuje drzewo do pliku binarnego w kolejnosci preorder.

Parameters

tree	Drzewo przeznaczone do zapisu.
fileName	Sciezka do pliku tworzonego lub nadpisywanego przy zapisie.

Returns

`true` po pomyslnym zapisie albo `false` w razie bledu otwarcia, zapisu lub zamkniecia pliku.

Najpierw zapisuje liczbe wezlow jako `std::uint64_t`, a nastepnie wartosci jako `int`.

Note

Format zalezy od rozmiaru typu `int` i kolejnosci bajtow platformy.

Sprawdzenie stanu po zamknieciu uwzglednia bledy zapisu bufora.

The documentation for this class was generated from the following files:

- BSTFile.h
- BSTFile.cpp

4.3 Node Struct Reference

Wezel drzewa przechowywanego w kontenerze STL.

`#include <BST.h>`

Public Member Functions

- Node (int value)

  Tworzy wezel bez dzieci.

Public Attributes

- int **value**

  Wartosc przechowywana w wezle.
- int **left**

  Indeks lewego dziecka albo -1.
- int **right**

  Indeks prawego dziecka albo -1.

4.3.1 Detailed Description

Wezel drzewa przechowywanego w kontenerze STL.

Pola left i right zawieraja indeksy dzieci w wektorze. Wartosc -1 oznacza brak dziecka.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 Node()

```
Node::Node (
            int value)  [inline]
```

Tworzy wezel bez dzieci.

Parameters

value	Wartosc nowego wezla.

The documentation for this struct was generated from the following file:

- BST.h

Chapter 5

File Documentation

5.1 BST.cpp File Reference

Implementacja binarnego drzewa poszukiwan przechowywanego w wektorze.

```
#include "BST.h"
#include <iostream>
#include <fstream>
```

5.1.1 Detailed Description

Implementacja binarnego drzewa poszukiwan przechowywanego w wektorze.

See also

BST

5.2 BST.h File Reference

Deklaracje wezla i klasy binarnego drzewa poszukiwan.

```
#include <cstddef>
#include <vector>
#include <string>
```

Classes

- struct Node

  Wezel drzewa przechowywanego w kontenerze STL.
- class BST

  Binarne drzewo poszukiwan oparte na std::vector.

5.2.1 Detailed Description

Deklaracje wezla i klasy binarnego drzewa poszukiwan.

5.3 BST.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef BST_H
00007 #define BST_H
00008
00009 #include <cstddef>
00010 #include <vector>
00011 #include <string>
00012
00019 struct Node {
00020     int value;
00021     int left;
00022     int right;
00023
00028     Node(int value) : value(value), left(-1), right(-1) {}
00029 };
00030
00037 class BST {
00038 private:
00039     std::vector<Node> tree;
00040     int rootIndex;
00041
00048     int insert(int currentIndex, int value);
00049
00054     void inorderHelper(int currentIndex);
00055
00060     void postorderHelper(int currentIndex);
00061
00071     std::size_t drawBranch(int currentIndex, std::size_t row, std::size_t& nextColumn,
00072                            std::vector<std::string>& lines) const;
00073
00074 public:
00076     BST();
00077
00084     bool saveToTextFile(const std::string& fileName) const;
00085
00091     bool addNode(int value);
00092
00108     bool removeNode(int value);
00109
00121     void removeTree();
00122
00124     void preorder();
00125
00127     void inorder();
00128
00130     void postorder();
00131
00136     std::vector<int> preorderValues() const;
00137
00142     void printTree() const;
00143
00149     bool findElement(int value) const;
00150
00159     std::vector<int> findPath(int value) const;
00160
00165
00166     void show();
00167
00168 };
00169
00170 #endif
```

5.4 BSTFile.cpp File Reference

Implementacja binarnego zapisu i odczytu drzewa oraz importu liczb z pliku tekstowego.

```
#include "BSTFile.h"
#include <cstdint>
#include <fstream>
#include <sstream>
#include <utility>
```

5.4.1 Detailed Description

Implementacja binarnego zapisu i odczytu drzewa oraz importu liczb z pliku tekstowego.

See also

BSTFile

5.5 BSTFile.h File Reference

Deklaracje operacji plikowych dla binarnego drzewa poszukiwan.

```
#include "BST.h"
#include <string>
```

Classes

- class BSTFile

  Udostepnia binarny zapis i odczyt drzewa oraz import liczb z pliku tekstowego.

5.5.1 Detailed Description

Deklaracje operacji plikowych dla binarnego drzewa poszukiwan.

5.6 BSTFile.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef BST_FILE_H
00007 #define BST_FILE_H
00008
00009 #include "BST.h"
00010 #include <string>
00011
00013 class BSTFile {
00014 public:
00025     static bool saveBinary(const BST& tree, const std::string& fileName);
00026
00035     static bool loadBinary(BST& tree, const std::string& fileName);
00036
00046     static bool loadText(BST& tree, const std::string& fileName);
00047 };
00048
00049 #endif
```

5.7 main.cpp File Reference

Interfejs konsolowy do obslugi binarnego drzewa poszukiwan.

```
#include "BST.h"
#include "BSTFile.h"
#include <iostream>
#include <sstream>
#include <string>
```

Functions

- int main ()

  Uruchamia menu konsolowe do edycji, wyszukiwania i wyswietlania drzewa.

5.7.1 Detailed Description

Interfejs konsolowy do obslugi binarnego drzewa poszukiwan.

5.7.2 Function Documentation

5.7.2.1 main()

```
int main ()
```

Uruchamia menu konsolowe do edycji, wyszukiwania i wyswietlania drzewa.

Menu obsluguje takze zapisywanie i wczytywanie plikow. W trybie dodawania lub usuwania wpisanie `q` przywraca menu glowne.

Returns

`0` po wybraniu opcji wyjscia lub gdy nie mozna dalej czytac wejscia.

Odczyt numeru opcji wymaga, aby caly token reprezentowal jedna liczbe calkowita.

Odczyt calego wiersza pozwala podac sciezke zawierajaca spacje.

Index

addNode
    BST, 8

BST, 7
    addNode, 8
    findElement, 8
    findPath, 8
    preorderValues, 9
    printTree, 9
    removeNode, 9
    removeTree, 10
    saveToTextFile, 10
    show, 11
BST.cpp, 15
BST.h, 15
BSTFile, 11
    loadBinary, 12
    loadText, 12
    saveBinary, 12
BSTFile.cpp, 16
BSTFile.h, 17

findElement
    BST, 8
findPath
    BST, 8

loadBinary
    BSTFile, 12
loadText
    BSTFile, 12

main
    main.cpp, 18
main.cpp, 18
    main, 18

Node, 13
    Node, 14

preorderValues
    BST, 9
printTree
    BST, 9

README, 1
removeNode
    BST, 9
removeTree
    BST, 10

saveBinary
    BSTFile, 12
saveToTextFile
    BST, 10
show
    BST, 11